Scale in Distributed Systems

Observation

Many developers of modern distributed systems easily use the ac
"scalable" without making clear **why** their system actually scale

---

Scalability

At least three components:

- Number of users and/or processes (size scalability)
- Maximum distance between nodes (geographical scalability)
- Number of administrative domains (administrative scalability)

Observation

Most systems account only, to a certain extent, for size scalability
(non)solution: powerful servers. Today, the challenge lies in
geographical and administrative scalability.

Geographical scalability

Cannot simply go from LAN to WAN:	WAN links are often inherently unreliable	Lack of m commun
• Many distributed systems assume synchronous client-server interactions • Client sends request and waits for an answer. • Latency may easily prohibit this scheme.	• Simply moving streaming video from LAN to WAN will fail.	• A simple broadcas cannot b deployed • Solution Develop separate and dire services their own scalabili problems

Administrative scalability

Essence: Conflicting policies concerning usage (and th payment), management, and security

Examples:

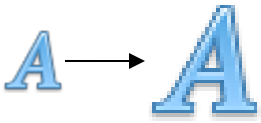
- Computational grids: share expensive resources between different domains.
- Shared equipment: how to control, manage, and use a shared radio telescope constructed as large-scale shared sensor network?

Exception: Several pee peer networks

- File-sharing systems (based, e.g., on BitT
- Peer-to-peer telepho (Skype)
- Peer-assisted audio streaming (Spotify)

Note: End users collab and not administrative entities.

Size Scalability



- Root causes for scalability problems with centralized solutions

Computational capacity, limited by the CPUs

Storage capacity, including the transfer rate between CPUs and disks

Network between the user and the centralized service

Size Scaling Technique 1

Hide communication latencies

Avoid waiting for responses; do something else:

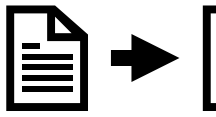
Make use of asynchronous communication

Have separate handler for incoming response

Problem: not every application fits this model

Size Scaling Technique 2

Replication/caching



Make copies of data available at different machines:

Replicated file servers and databases

Mirrored Web sites

Web caches (in browsers and proxies)

File caching (at server and client)

Replication Problems

Observation

Applying scaling techniques is easy, except for one thi

- Having multiple copies (cached or replicated), leads inconsistencies: modifying one copy makes that cop different from the rest.
- Always keeping copies consistent and in a general v requires global synchronization on each modificatio
- Global synchronization precludes large-scale soluti

Observation

If we can tolerate inconsistencies, we may reduce the for global synchronization, but tolerating inconsisten application dependent.

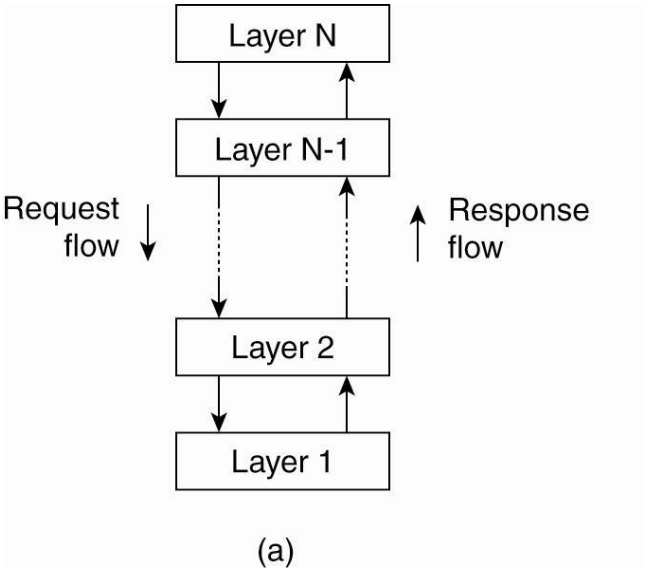
So Far

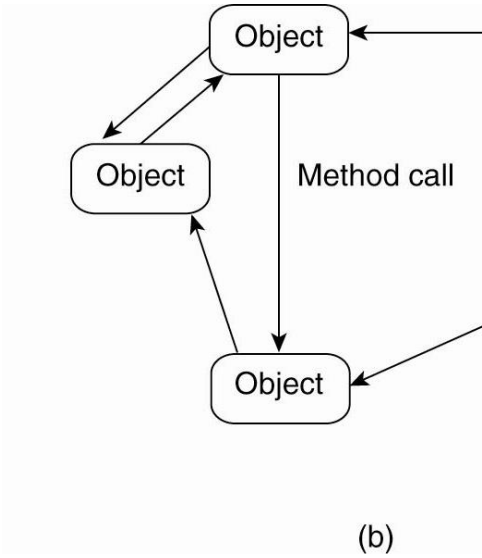
- Definitions and Goals
- Architectural Styles
- System Architectures
- Peer to Peer
- Edge Computing
- Blockchain basics

Architectural Styles

Basic Idea

Organize into logically different components and distrib
those components over the various machines





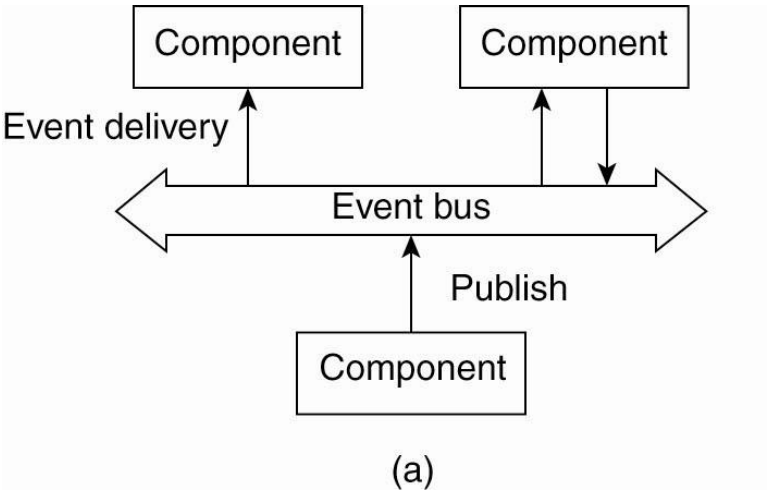
(a) Layered style is used for client-server system

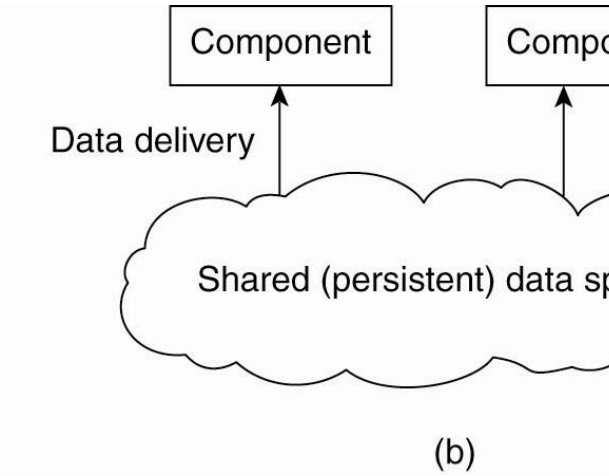
(b) Object-based style for distributed object systems

Architectural Styles

Observation

Decoupling processes (“anonymous”) and removing ti
constraints (“asynchronous”) led to alternative style





(a) Publish/Subscribe [anonymous]

(b) Shared dataspace [anonymous and asynchronous]

So Far

- Definitions and Goals
- Architectural Styles
- System Architectures
- Peer to Peer
- Edge Computing
- Blockchain basics